

Özel Bellek Yöneticisi

Custom Memory Allocator

Sistem Programlama — Dönem Sonu Projesi

Standart **malloc** / **free** / **calloc** mantığını taklit eden, kullanıcı alanında çalışan, thread-safe bir bellek yöneticisi

Ekip Üyesi	Rol	Sorumluluk Alanı
Demirhan Çakır	Kişi 1 — Çekirdek	Blok header, sbrk havuzu, free list, pointer aritmetiği
Mert Baytaş	Kişi 2 — İşlemler	first-fit/best-fit, splitting, free, coalescing, calloc, hata kontrolü
Büşra Ay	Kişi 3 — Sistem	Mutex/thread-safety, fragmentation raporu, test, Makefile, TUI

Proje Deposu (GitHub)

<https://github.com/baytasmert/sistem-programlama-bellek-yonetimi>

Tarih: Haziran 2026 • Dil: C (C11) • Platform: POSIX / Linux

1. Amaç

Bu projenin amacı, modern işletim sistemlerinde çekirdek tarafından sağlanan dinamik bellek yönetiminin (**malloc / free / calloc**) kullanıcı alanında, sıfırdan ve eğitim amaçlı olarak yeniden inşa edilmesidir. Geliştirilen **my_malloc**, **my_free** ve **my_calloc** fonksiyonları; işletim sisteminden **sbrk** çağrısıyla alınan tek parça bir bellek havuzunu kendi veri yapıları üzerinden yöneterek standart kütüphane davranışını taklit eder.

Proje; serbest blok listesi yönetimi, yerleştirme stratejileri (first-fit / best-fit), blok bölme (splitting) ve birleştirme (coalescing) ile fragmentation kontrolü, magic number tabanlı bütünlük denetimi, double-free ve geçersiz pointer tespiti, bellek sızıntısı raporlaması ve **POSIX thread**'lerle eşzamanlı erişimde **mutex** tabanlı senkronizasyon konularını bir arada ele alır. Böylece düşük seviyeli bellek yönetimi, işaretçi aritmetiği, veri yapıları ve thread güvenliği kavramları tek bir uygulamada bütünleşik olarak gösterilmiştir.

1.1. Zorunlu Gereksinimlerin Karşılanması

Aşağıdaki tablo, proje tanımındaki zorunlu maddelerin hangi modülde ve nasıl karşılandığını özetler. Tüm zorunlu maddeler eksiksiz olarak gerçekleştirilmiştir.

Zorunlu Gereksinim	Durum	Karşılandığı Yer
my_malloc, my_free, my_calloc	☐ Tam	allocator_ops.c (+ threadsafe sarmalayıcı)
Serbest blok listesi (free list)	☐ Tam	allocator_core.c — çift bağlı liste
Yerleştirme stratejisi (first-fit / best-fit)	☐ İkisi de	allocator_ops.c
Fragmentation hesaplama ve raporlama	☐ Tam	allocator_debug.c — print_memory_stats()
Bellek sızıntısı / double-free kontrolü	☐ Tam	allocator_safety.c, allocator_ops.c
Çok-threadli erişimde mutex	☐ Tam	allocator_threadsafe.c — pthread_mutex
Test programı ile doğrulama	☐ Tam	tests/test_allocator.c

Ortak kurallar açısından da proje; **C (C11)** ve **POSIX/Linux API** (sbrk, pthread) kullanır, **thread** ve **mutex** ile senkronizasyon içerir, hem stderr'e hem **allocator_test.log** dosyasına **loglama** yapar, **perror** ve özel hata mesajlarıyla **hata yönetimi** sağlar ve ölçülmüş bir **performans değerlendirmesi** sunar (Bölüm 7).

2. Tasarım

2.1. Modüler Mimari

Sistem, sorumlulukları net biçimde ayrılmış modüllerden oluşur. Üst katmandaki genel API (my_malloc/my_free/my_calloc) çağrıları önce thread-safe sarmalayıcıdan geçer, oradan çekirdek tahsis mantığına ve en altta işletim sistemine iner.

```
Kullanici Uygulamasi / Test / Terminal UI
| my_malloc() my_free() my_calloc()
v
[allocator_threadsafe.c] <- Kisi 3 : tek global mutex (kilit al/birak)
| allocator_raw_malloc/free/calloc
v
[allocator_ops.c] <- Kisi 2 : strateji, splitting, coalescing
[allocator_safety.c] <- Kisi 2 : magic, double-free, leak raporu
| free list + all-blocks listesi API'si
v
[allocator_core.c] <- Kisi 1 : init, liste yönetimi, pointer aritmetigi
| sbrk()
v
Isletim Sistemi (heap / program break)
```

Şekil 1 — Katmanlı mimari ve modül–kişi eşleşmesi

Dikkat çeken bir tasarım kararı: thread-safety, mevcut tahsis mantığını *hiç değiştirmeden* eklenmiştir. Makefile, `allocator_ops.c` derlenirken `my_malloc/my_free/my_calloc` sembollerini `-D` ile `allocator_raw_malloc/raw_free/raw_calloc` olarak yeniden adlandırır. Genel (public) isimleri ise `allocator_threadsafe.c` yeniden tanımlar; bu sarmalayıcı kilidi alır, ham fonksiyonu çağırır ve kilidi bırakır. Böylece kilitleme katmanı, çekirdek koda dokunmadan temiz bir biçimde araya girer.

2.2. Bellek Düzeni ve Blok Header

Havuzdan ayrılan her blok **[Header][Payload]** düzenindedir. Kullanıcıya yalnızca payload adresi döndürülür; header hemen önünde gizli kalır. Header, 16 byte hizalamayı koruyacak şekilde padding ile doldurulur ve bütünlük için bir **magic** imzası taşır.

```
struct block_header {
    size_t      size;           // kullanıcıya acılan payload boyutu
    int         is_free;        // 1 = serbest, 0 = tahsisli
    uint32_t    magic;          // 0xC0FFEE01 (alloc) / 0xC0FFEE02 (free)
    block_header_t *next_free; // \ serbest blok listesi (cift bagli)
    block_header_t *prev_free; // /
    block_header_t *next_all;  // \ heap sirali tum-blok listesi (cift bagli)
    block_header_t *prev_all;  // /
    uint8_t     padding[...]; // 16-byte hizalama icin
};
```

Bellek: +-----+
| block_header | payload (kullanici) |
+-----+
^ ^
payload_to_block block_to_payload (ptr +/- 1 header)

Şekil 2 — Blok header yapısı ve payload ilişkisi

2.3. İki Bağlı Liste

- **Serbest blok listesi (free list):** Yalnızca boş bloklar; yerleştirme stratejileri bu listeyi tarar. Başa ekleme ile $O(1)$ ekleme, çift bağlı yapı ile $O(1)$ çıkarma.
- **Tüm-blok listesi (all-blocks):** Bellekte adres sırasına göre tüm bloklar. Coalescing (fiziksel komşuluk), fragmentation hesabı ve sızıntı raporu bu liste üzerinden çalışır.

2.4. Yerleştirme Stratejileri

İki strateji de gerçekleşmiş ve `allocator_set_strategy()` ile çalışma zamanında değiştirilebilir biçimde tasarlanmıştır (Strategy deseni).

Kriter	First-Fit (varsayılan)	Best-Fit
Seçim	İlk yeterli blok	En küçük yeterli blok
Tarama	Erken çıkış (çoğunlukla kısa)	Tüm listeyi gezer
Hız	Daha hızlı	Görece yavaş
Fragmentation	Daha yüksek olabilir	Daha düşük (daha verimli)

2.5. Splitting ve Coalescing

Seçilen blok istenenden büyükse, fazlalık ayrı bir serbest bloğa **bölünür (splitting)** ve internal fragmentation azalır. Serbest bırakmada, all-blocks listesindeki fiziksel komşular boşsa **birleştirilir (coalescing)** ve external fragmentation azalır. Bölme yalnızca kalan parça bir header + minimum blok boyutunu taşıyabiliyorsa yapılır; aksi halde gereksiz parçalanma önlenir.

```
SPLITTING (my_malloc icinde):
  once: [Header][===== 512 byte payload =====]
  sonra: [Header][== 256 ==][Header][== kalan serbest ==]

COALESCING (my_free icinde, onceki + sonraki komsu kontrol edilir):
  once: [ serbest ][ SERBEST BIRAKILAN ][ serbest ]
  sonra: [===== tek serbest blok =====]
```

Şekil 3 — Bölme ve birleştirme ile fragmentation kontrolü

Bu davranış testlerde doğrulanmıştır: program sonunda tüm bloklar serbest bırakıldığında, coalescing sayesinde havuz tek bir **65536 byte**'lık serbest bloğa geri birleşmektedir (Bölüm 6 çıktısı).

2.6. Thread-Safety Tasarımı

Tüm genel API çağrıları tek bir global **pthread_mutex_t allocator_mutex** ile korunur. Kilit alma/bırakma hataları **strerror** ile raporlanır. **my_free(NULL)** ve calloc taşma kontrolü gibi durumlar, gereksiz kilitlenmeyi önlemek için kilit alınmadan önce ele alınır. Raporlama fonksiyonları (`print_memory_stats`, TUI istatistikleri) de listeyi okurken aynı kilidi tutar; böylece eşzamanlı değişiklik sırasında tutarsız veri okunmaz.

3. Kullanılan Sistem Programlama Kavramları

3.1. İşletim Sistemi Bellek İsteği — `sbrk`

Program break, **sbrk(0)** ile okunur ve **sbrk(boyut)** ile ileri taşınarak heap büyütülür. Havuz tek parça istenir; allocator bu ham bölgeyi kendi blok yapılarına böler. Havuz yetmediğinde **allocator_request_from_os()** ile ek bölge alınır.

3.2. İşaretçi Aritmetiği ve Hizalama

Header ile payload arasındaki geçiş (**block_header_t ***)`ptr - 1` ve **block + 1** aritmetiğiyle yapılır. Tüm payload adresleri **16 byte** sınırına hizalanır; istenen boyutlar bit maskesi ile yukarı yuvarlanır ve taşma (overflow) kontrol edilir.

3.3. POSIX Thread ve Senkronizasyon

pthread_create / **pthread_join** ile birden çok iş parçacığı oluşturulur; ortak veri yapısı **pthread_mutex_lock/unlock** ile korunur. Test programı 8 thread ile eşzamanlı binlerce malloc/free yaparak veri yapısının bozulmadığını gösterir.

3.4. Bellek Bütünlüğü — Magic Number

Her header bir **magic** imzası taşır (0xC0FFEE01 tahsisli, 0xC0FFEE02 serbest). Geçersiz pointer veya bellek bozulması bu imza ile yakalanır; double-free ise **is_free** bayrağı ile tespit edilir.

3.5. Loglama ve Hata Yönetimi

Hatalar **stderr**'e açıklayıcı Türkçe mesajlarla yazılır; sistem çağrısı hataları **perror** / **strerror(errno)** ile raporlanır. Test programı tüm adımları ayrıca **allocator_test.log** dosyasına yazarak kalıcı kayıt tutar.

4. Çalıştırma Adımları

Proje Linux / POSIX ortamında **GNU Make** ile derlenir. Bağımlılıklar standart C kütüphanesi ve **pthread**'dir. Windows üzerinde **WSL** (Ubuntu) ile sorunsuz çalışır.

```
# 1) Derleyici araclari (Debian/Ubuntu)
sudo apt update && sudo apt install build-essential

# 2) Derleme
make          # projeyi derler -> ./test_allocator

# 3) Testler (terminal arayuzu acmadan, CI/otomasyon dostu)
make test     # ./test_allocator --no-ui

# 4) Interaktif ANSI terminal arayuzu (canli demo, bellek haritasi)
make run      # once testler, sonra TUI

# 5) Temizlik
make clean    # build/ ciktilari ve allocator_test.log silinir
```

Makefile hedefleri: **all** (derle), **test** (otomatik testler), **run** (TUI), **clean** (temizlik), **help** (komut listesi). Derleme bayrakları **-Wall -Wextra -std=c11 -O2 -pthread** ile sıkı uyarı ve optimizasyon kullanır.

5. Testler

Test programı (**tests/test_allocator.c**) dört kategori altında doğrulama yapar ve her adımı ekrana + log dosyasına yazar. Başarısız bir koşul programı **[FAIL]** ile sonlandırır.

Kategori	Kapsanan Senaryolar
Temel testler	malloc(64)+memset, calloc(16,int) ve tüm elemanların sıfırlandığının doğrulanması
Edge case / hata	0 byte malloc → NULL, SIZE_MAX malloc → güvenli ret, free(NULL), double-free
Thread-safety + perf	8 thread × 2000 iterasyon eşzamanlı malloc/free; süre ölçümü
Fragmentation	Ortak blok serbest bırakılıp print_memory_stats() ile rapor

Aşağıda, projenin **WSL2 / Ubuntu (gcc 13.3, aarch64, 12 çekirdek)** üzerinde gerçek çalışma çıktısı yer alır (tekrarlayan calloc satırları kısaltılmıştır):

```
[TEST] Temel malloc/free/calloc
[OK] my_malloc 64 byte alan dondurdu
[OK] my_calloc alan dondurdu
[OK] my_calloc belleği sıfırladı          (16 elemanın tamamı doğrulandı)

[TEST] Edge case ve hata senaryolari
[OK] 0 byte malloc NULL dondurdu
[OK] Çok büyük malloc güvenli şekilde reddedildi
[OK] NULL free sessizce yok sayıldı
[OK] double-free testi için bellek alındı
[OK] double-free stderr uyarısı ile yakalandı
    -> stderr: 'zaten serbest olan blok tekrar serbest bırakılmaya çalışıldı (double-free)'

[TEST] Thread safety ve performans
[OK] pthread_create başarılı      (x8)
[OK] pthread_join başarılı       (x8)
[PERF] 8 thread x 2000 iterasyon: 6134 us

[TEST] Fragmentation senaryosu
[OK] fragmentation blokları ayrıldı
```

Çıktı 1 — Otomatik test akışı (make test)

Fragmentation senaryosunda ortak blok serbest bırakıldıktan sonraki istatistik ve program sonunda tüm bloklar bırakıldıktan sonraki **coalescing** sonucu:

```
=== Bellek Yöneticisi İstatistikleri (ortadaki blok bos) ===
Toplam rezerve bellek : 65600 byte (64.06 KB)
Kullanımdaki payload : 512 byte
Serbest blok sayısı : 2 En büyük serbest blok : 64320 byte
External fragmentation : 0.79% Toplam kullanım oranı : 0.78%

=== Program sonu (tüm bloklar serbest -> COALESCING) ===
Serbest blok sayısı : 1 En büyük serbest blok : 65536 byte
External fragmentation : 0.00% <- havuz tek bloğa geri birleştirdi

=== Bellek Sızıntısı Raporu ===
Sızıntı tespit edilmedi - Tümü temizlendi!
```

Çıktı 2 — Fragmentation, coalescing ve sızıntı raporu

Sonuç: tüm testler **başarılı**, double-free **yakalandı**, eşzamanlı erişimde **çökme yok** ve program sonunda **sızıntı yok**. Coalescing'in havuzu tek bloğa indirmesi, birleştirmenin doğru çalıştığının somut kanıtıdır.

6. Performans Değerlendirmesi

Performans, sabit toplam iş yükü (**200.000** malloc/free çifti) thread sayısına bölünerek ölçülmüştür. İlk koşu ısınma (warm-up) olarak elenmiş, sonraki iki kararlı koşunun ortalaması alınmıştır. Ortam: WSL2 Ubuntu, aarch64, 12 çekirdek, gcc 13.3 -O2.

Thread	Süre (ms)	Verim (Mop/s)	Görelî hız
1	14.0	14.25	1.00x (taban)
2	23.6	8.46	0.59x
4	31.8	6.30	0.44x
8	37.7	5.30	0.37x
16	59.7	3.35	0.24x

Tablo — Tek global mutex altında thread sayısına göre verim (200K işlem)

6.1. Yorum

Tek bir global mutex tüm tahsis/serbest işlemlerini **serileştirir**. Bu nedenle thread sayısı arttıkça toplam verim artmaz; aksine **kilit çekişmesi (lock contention)** ve bağlam değiştirme maliyeti yüzünden bir miktar **azalır**. Tek thread en yüksek verimi verir (~14,2 milyon işlem/sn). Bu, doğruluğu garanti eden ancak ölçeklenmeyi sınırlayan bilinçli bir tasarım tercihidir; ölçütümüz bu davranışı net biçimde doğrular.

Önemli nokta: amaç ham ölçeklenme değil, **çok-threadli ortamda veri yapısının bozulmadan kalmasıdır**. Stres testinde 8 thread × 2000 iterasyon hiçbir çökme/bozulma olmadan tamamlanır. Ölçeklenmeyi artırmak gelecek çalışmadır (Bölüm 8).

7. Karşılaşılan Problemler ve Çözümler

7.1. Splitting'de İşaretçi Aritmetiği Hatası

Blok bölme sırasında yeni header adresi hesaplanırken fazladan bir - 1 kullanılıyordu; bu, header'ı bir header boyutu kadar yanlış konuma yazarak bellek düzenini bozuyordu. Hata kod incelemesinde yakalandı ve **new_free_block = (block_header_t *)split_point;** şeklinde düzeltildi. Doğrulama için splitting/coalescing senaryoları yeniden çalıştırıldı.

7.2. Thread-Safety'i Çekirdek Koda Dokunmadan Ekleme

Mevcut **my_malloc/my_free/my_calloc** gövdelerini değiştirmeden kilit eklemek gerekiyordu. Çözüm, Makefile'da **-D** ile sembol yeniden adlandırma yapıp genel isimleri ayrı bir sarmalayıcı dosyada (allocator_threadsafe.c) mutex ile yeniden tanımlamak oldu. Bu, modüller arası bağımlılığı azaltan temiz bir

ayırıştırma sağladı.

7.3. Kilit Çekişmesi ve Ölçeklenme

Tek global kilit, çok-threadli verimi sınırlıyor (Bölüm 6). Bu projede doğruluk önceliklendirildi; ölçeklenme bilinçli olarak ikinci planda bırakıldı ve ölçümlerle şeffafla raporlandı.

7.4. Türkçe Karakter ve ANSI TUI Hizalaması

Terminal arayüzünde Türkçe karakterler (çok baytlı UTF-8) ve ANSI renk kodları, çerçeve genişliğini bozuyordu. Çözüm, yalnızca ekranda görünen karakterleri sayan bir `utf8_width()` yardımcı fonksiyonu ve renk kodlarını genişlik hesabına katmamak oldu.

7.5. Platform Farkı (Windows / Linux)

Allocator `sbrk` ve `pthread` kullandığından doğrudan Windows/MinGW ile derlenmez. Geliştirme ve testler **WSL (Ubuntu)** üzerinde yapıldı; bu, projenin POSIX hedefiyle tutarlıdır.

8. Sonuç ve Gelecek Çalışmalar

Proje, standart dinamik bellek yönetiminin temel mekanizmalarını kullanıcı alanında başarıyla yeniden üretmiştir. Zorunlu gereksinimlerin tamamı karşılanmış; first-fit/best-fit stratejileri, splitting/coalescing ile fragmentation kontrolü, magic tabanlı hata tespiti, sızıntı raporu, mutex tabanlı thread-safety ve kapsamlı bir test/benchmark altyapısı bir arada gerçekleşmiştir. Tüm testler gerçek bir Linux ortamında geçmiş, double-free ve sızıntılar doğru tespit edilmiş, coalescing'in havuzu tek bloğa indirilmesi gözlemlenmiştir.

Gelecek çalışmalar: (1) tek global kilit yerine **arena / thread-local önbellek** (tcmalloc/jemalloc benzeri) ile ölçeklenebilir kilitleme; (2) **realloc** desteği; (3) boyut sınıfları (size classes) ile küçük tahsislerde daha düşük internal fragmentation; (4) serbest blokların adres sıralı tutulmasıyla daha hızlı coalescing.

Ek — Kaynak Dosya Yapısı

Dosya	Satır	Sahip	Açıklama
include/allocator.h	114	Kişi 1	Ortak header: struct, sabitler, bildirimler
src/allocator_core.c	410	Kişi 1	init, free list, all-blocks, pointer aritmetiği, sbrk
src/allocator_ops.c	300	Kişi 2	first/best-fit, splitting, coalescing, malloc/free/calloc
src/allocator_safety.c	150	Kişi 2	magic doğrulama, double-free, sızıntı raporu
src/allocator_threadsafe.c	98	Kişi 3	mutex sarmalayıcı (public API)
src/allocator_debug.c	69	Kişi 3	print_memory_stats — fragmentation
src/allocator_terminal_ui.c	829	Kişi 3	ANSI terminal arayüzü (bonus)
tests/test_allocator.c	203	Kişi 3	test + thread + performans + log
Makefile	61	Kişi 3	derleme, sembol yeniden adlandırma, hedefler
TOPLAM	2234	—	kaynak + test + yapı